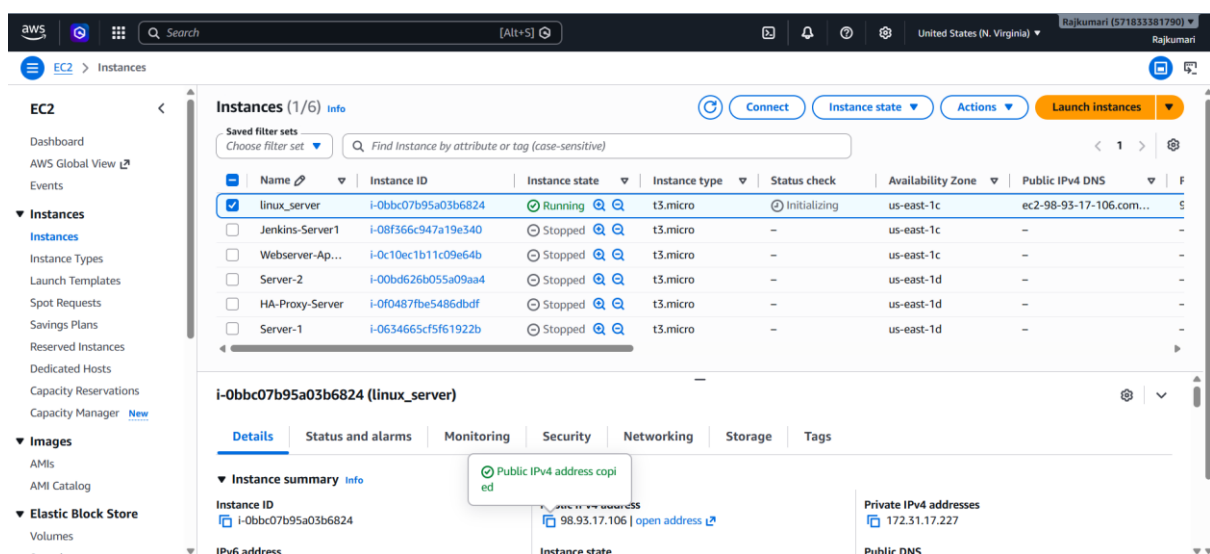


1. Create a bash script to check if a directory is available or not.

Objective -> To create a Bash script that checks whether a specified directory exists on the system.

Purpose -> The purpose of this script is to verify the availability of a directory before performing operations such as creating, reading, writing, or copying files. This helps prevent errors caused by referencing a non-existent directory.

Step: 1 First, Connect with AWS EC2 Instance by using .pem and ip address file in git bash



Explanation

read -p: Accepts the directory path from the user.

-d: Directory test operator. It checks whether the given path exists and is a directory.

if: Evaluates the condition.

then: Executes the statements if the condition is true.

else: Executes the statements if the condition is false.

fi: Ends the if statement.

```
brajk@RajKumari MINGW64 ~/Downloads  
$ ssh -i linux_key.pem ec2-user@98.93.17.106
```

Step:2 Create a new script file using the vi editor.

vi dir.bash

cat dir.bash

```
[root@ip-172-31-17-227 bashscript]# cat dir.bash  
#!/bin/bash  
  
# Read directory name from user  
read -p "Enter the directory: " dir  
  
# Check if directory exists  
if [ -d "$dir" ]  
then  
    echo "Directory exists."  
else  
    echo "Directory does not exist."  
fi  
[root@ip-172-31-17-227 bashscript]# |
```

Step:3 Type the Script and save & exit

```
#!/bin/bash  
  
# Read directory name from user  
read -p "Enter the directory: " dir  
  
# Check if directory exists  
if [ -d "$dir" ]  
then  
    echo "Directory exists."  
else  
    echo "Directory does not exist."  
fi
```

Step:4 Give Execute Permission (# chmod 755 dir.bash)

Step:5 Execute the Script (# ./dir.bash)

```
[root@ip-172-31-17-227 bashscript]# vi dir.bash
[root@ip-172-31-17-227 bashscript]# ./dir.bash
Enter the directory: raji
Directory does not exist.
[root@ip-172-31-17-227 bashscript]# ./dir.bash
Enter the directory: raj
Directory exists.
[root@ip-172-31-17-227 bashscript]# |
```

2. Create a bash script to create multiple files.

Objective: To create a Bash script that creates multiple files automatically based on the user's input.

Purpose: The purpose of this script is to automate the creation of multiple files, reducing manual effort and saving time. It is useful for creating test files, log files, configuration files, or project files in bulk.

```
[root@ip-172-31-17-227 bashscript]# vi multiple.sh
[root@ip-172-31-17-227 bashscript]# vi multiple.sh
[root@ip-172-31-17-227 bashscript]# chmod 755 multiple.sh
[root@ip-172-31-17-227 bashscript]# cat multiple.sh
#!/bin/bash
```

```
read -p "Enter the number of files to create: " num
for ((i=1; i<=num; i++))
do
    touch file$i.txt
done
echo "$num files created successfully."
```

```
#!/bin/bash
read -p "Enter the number of files to create: " num
for ((i=1; i<=num; i++))
do
    touch file$i.txt
done
echo "$num files created successfully."
~
```

```
[root@ip-172-31-17-227 bashscript]# ./multiple.sh
Enter the number of files to create: 6
6 files created successfully.
[root@ip-172-31-17-227 bashscript]# ls
dir.bash file1.txt file2.txt file3.txt file4.txt file5.txt file6.txt multiple.sh raj
[root@ip-172-31-17-227 bashscript]#
```

Explanation

read -p: Accepts the number of files from the user.

for ((i=1; i<=num; i++)): Runs the loop from 1 to the number entered.

touch file\$i.txt: Creates files named file1.txt, file2.txt, file3.txt, etc.

echo: Displays a success message after creating the files.

3. Create a bash script to take a backup of a directory.

Objective: To create a Bash script that takes a backup of a specified directory by compressing it into a .tar.gz archive.

Purpose: The purpose of this script is to create a compressed backup of a directory, helping protect important data from accidental deletion, corruption, or system failure. **The backup file can be stored or transferred for future recovery.**

```
#!/bin/bash

read -p "Enter the directory name to back up: " dir

if [ -d "$dir" ]
then
    backup_name=$(basename "$dir")_$(date +%Y%m%d_%H%M%S).tar.gz
    tar -czf "$backup_name" "$dir"
    echo "Backup created successfully: $backup_name"
else
    echo "Directory does not exist."
fi
```

```
[root@ip-172-31-17-227 bashscript]# ./backup.sh
Enter the directory path to back up: raj
Backup created successfully: raj_20260705_062716.tar.gz
[root@ip-172-31-17-227 bashscript]# ls
backup.sh  file1.txt  file3.txt  file5.txt  multiple.sh  raj_20260705_062716.tar.gz
dir.bash  file2.txt  file4.txt  file6.txt  raj
```

Explanation

- **read -p** – Accepts the directory path from the user.
- **-d** – Checks whether the entered path is an existing directory.
- **basename** – Extracts only the directory name from the full path.
- **date +%Y%m%d_%H%M%S** – Adds the current date and time to make the backup filename unique.
- **tar -czf** – Creates a compressed .tar.gz archive.
 - **-c** → Create a new archive.
 - **-z** → Compress the archive using gzip.
 - **-f** → Specify the output filename.
- **echo** – Displays a success or error message.

4.Create a bash script to install Nginx on an EC2 server.

Objective: To create a Bash script that installs and starts the Nginx web server on an AWS EC2 Linux instance.

Purpose: The purpose of this script is to automate the installation, startup, and enabling of the Nginx web server. This reduces manual effort, ensures consistency, and prepares the EC2 instance to host web applications.

```
#!/bin/bash

echo "Installing Nginx..."

dnf install -y nginx

systemctl start nginx

systemctl enable nginx

systemctl status nginx --no-pager

echo "Nginx installation completed successfully."
~

libunwind-1.4.0-5.amzn2023.0.3.x86_64          nginx-1:1.30.2-1.amzn2023.0.1.x86_64
nginx-core-1:1.30.2-1.amzn2023.0.1.x86_64      nginx-filesystem-1:1.30.2-1.amzn2023.0.1.noarch
nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-07-05 06:37:23 UTC; 259ms ago
     Main PID: 12642 (nginx)
       Tasks: 3 (limit: 1059)
      Memory: 3.5M
         CPU: 62ms
    CGroup: /system.slice/nginx.service
            └─12642 "nginx: master process /usr/sbin/nginx"
              └─12643 "nginx: worker process"
                └─12644 "nginx: worker process"

Jul 05 06:37:23 ip-172-31-17-227.ec2.internal systemd[1]: Starting nginx.service - The nginx HTTP ...
Jul 05 06:37:23 ip-172-31-17-227.ec2.internal nginx[12639]: nginx: the configuration file /etc/ngi...s ok
Jul 05 06:37:23 ip-172-31-17-227.ec2.internal nginx[12639]: nginx: configuration file /etc/nginx/n...sful
Jul 05 06:37:23 ip-172-31-17-227.ec2.internal systemd[1]: Started nginx.service - The nginx HTTP a...ver.
Hint: Some lines were ellipsized, use -l to show in full.
Nginx installation completed successfully.
[root@ip-172-31-17-227 bashscript]#
```

```
[root@ip-172-31-17-227 bashscript]# ss -tulnp | grep nginx
tcp    LISTEN 0      511                0.0.0.0:80          0.0.0.0:*        users:(("nginx",pid=12644,fd=8),("nginx",pid=12642,fd=8))
tcp    LISTEN 0      511                [::]:80            [::]:*          users:(("nginx",pid=12644,fd=9),("nginx",pid=12643,fd=9),("nginx",pid=12642,fd=9))
[root@ip-172-31-17-227 bashscript]#
```

← → ↻ Not secure 98.93.17.106

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to fs.com/nginx.

Thank you for using nginx.

Explanation

- **#!/bin/bash** – Specifies that the script should run using the Bash shell.
- **dnf install -y nginx** – Installs the Nginx package without prompting for confirmation (Amazon Linux 2023)
- **systemctl start nginx** – Starts the Nginx service.
- **systemctl enable nginx** – Configures Nginx to start automatically after a reboot.
- **systemctl status nginx** – Displays the current status of the Nginx service.
- **echo** – Prints progress and success messages.

5. Create a bash script to install Apache Tomcat on an EC2 server.

Objective: To create a Bash script that installs Apache Tomcat on an AWS EC2 Linux instance, configures it as a service, and starts the Tomcat server.

Purpose: The purpose of this script is to automate the installation and configuration of Apache Tomcat on an EC2 server.

This ensures a consistent setup, reduces manual effort, and prepares the server for deploying Java web applications.

```
#!/bin/bash

echo "Installing Java..."
dnf install -y java-17-amazon-corretto wget tar

cd /opt

echo "Downloading Tomcat..."

wget https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.41/bin/apache-tomcat-10.1.41.tar.gz

if [ $? -ne 0 ]
then
    echo "Download failed."
    exit 1
fi

tar -xzf apache-tomcat-10.1.41.tar.gz

mv apache-tomcat-10.1.41 tomcat

chmod +x /opt/tomcat/bin/*.sh

/opt/tomcat/bin/startup.sh

echo "Tomcat installed successfully."
```

```
[root@ip-172-31-17-227 opt]# vi install_tomcat.sh
[root@ip-172-31-17-227 opt]# chmod 755 install_tomcat.sh
[root@ip-172-31-17-227 opt]# ./install_tomcat.sh
Installing Java...
Last metadata expiration check: 0:10:35 ago on Sun Jul  5 06:49:53 2026.
Package java-17-amazon-corretto-1:17.0.19+10-1.amzn2023.1.x86_64 is already installed.
Package wget-1.21.3-1.amzn2023.0.4.x86_64 is already installed.
Package tar-2:1.34-1.amzn2023.0.4.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
Downloading Tomcat...
--2026-07-05 07:00:28-- https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.41/bin/apache-tomcat-10.1.41.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f8:2b1:3000::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443: connected.
HTTP request sent, awaiting response... 200 OK
Length: 14079449 (13M) [application/x-gzip]
Saving to: 'apache-tomcat-10.1.41.tar.gz'

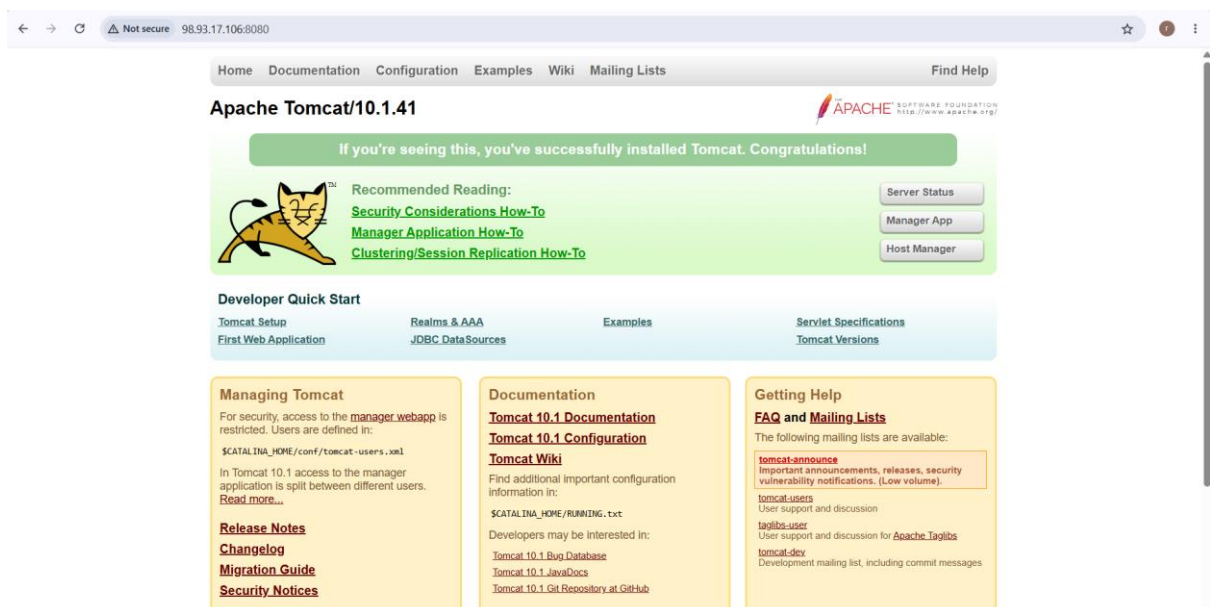
apache-tomcat-10.1.41.tar.gz                               100%[=====]
2026-07-05 07:00:30 (10.6 MB/s) - 'apache-tomcat-10.1.41.tar.gz' saved [14079449/14079449]

Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-jar.jar
Tomcat started.
Tomcat installed successfully.
[root@ip-172-31-17-227 opt]# |
```



```
[root@ip-172-31-17-227 opt]# ps -ef | grep tomcat
root      14202      1  4 07:00 pts/3    00:00:03 /usr/bin/java -Djava.util.logging.config.file=/opt/tomcat/conf/logging.properties -Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager -Djdk.tls.ephemeralDHKeySize=2048 -Djava.protocol.handler.pkgs=org.apache.catalina.webresources -Dsun.io.useC
anonCaches=false -Dorg.apache.catalina.security.SecurityListener.UMASK=0027 --add-opens=java.base/java.
lang=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-
UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNA
MED --add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED -classpath /opt/tomcat/bin/bootstrap.jar:/opt/to
mcat/bin/tomcat-juli.jar -Dcatalina.base=/opt/tomcat -Dcatalina.home=/opt/tomcat -Djava.io.tmpdir=/opt/
tomcat/temp org.apache.catalina.startup.Bootstrap start
root      14317    10382  0 07:02 pts/3    00:00:00 grep --color=auto tomcat
```

```
Created symlink /etc/systemd/system/multi-user.target.wants/tomcat.service → /etc/systemd/system/tomcat
service.
● tomcat.service - Apache Tomcat
   Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-07-05 07:14:22 UTC; 22ms ago
   Process: 15111 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
   Main PID: 15124 (java)
     Tasks: 2 (limit: 1059)
    Memory: 11.8M
       CPU: 33ms
   CGroup: /system.slice/tomcat.service
           └─15124 /usr/bin/java -Djava.util.logging.config.file=/opt/tomcat/conf/logging.properties
```



6. Create a bash script to check if the Nginx service is running, if not running then script should start the service.

Objective: To create a Bash script that checks whether the Nginx service is running. If the service is not running, the script automatically starts it.

Purpose: The purpose of this script is to monitor the status of the Nginx service and ensure it is always running.

If Nginx is stopped due to a system restart or an unexpected issue, the script will automatically start the service, helping maintain web server availability.

```
#!/bin/bash
SERVICE="nginx"
if systemctl is-active --quiet $SERVICE
then
    echo "Nginx service is already running."
else
    echo "Nginx service is not running."
    echo "Starting Nginx service..."
    systemctl start $SERVICE
    if systemctl is-active --quiet $SERVICE
    then
        echo "Nginx service started successfully."
    else
        echo "Failed to start Nginx service."
    fi
fi
~
~
~
```

```
[root@ip-172-31-17-227 bin]# vi nginx_service.sh
[root@ip-172-31-17-227 bin]# chmod +x nginx_service.sh
[root@ip-172-31-17-227 bin]# ./nginx_service.sh
Nginx service is already running.
[root@ip-172-31-17-227 bin]#
```

Explanation

- SERVICE="nginx" – Stores the service name in a variable.
- systemctl is-active --quiet nginx – Checks whether the Nginx service is running.
- if – Evaluates the service status.

- `systemctl start nginx` – Starts the Nginx service if it is stopped.
- `echo` – Displays status messages to the user

7. Create a bash script for a **calculator**.

Objective

To create a Bash script that performs basic arithmetic operations such as **Addition, Subtraction, Multiplication, and Division** based on the user's choice.

Purpose

The purpose of this script is to automate basic mathematical calculations. It accepts two numbers and an operator from the user, performs the selected operation, and displays the result.

```
#!/bin/bash

echo "Simple Calculator"

read -p "Enter first number: " num1
read -p "Enter second number: " num2

echo "Choose an operation:"
echo "1. Addition (+)"
echo "2. Subtraction (-)"
echo "3. Multiplication (*)"
echo "4. Division (/)"

read -p "Enter your choice (1-4): " choice

case $choice in
1)
    result=$((num1 + num2))
    echo "Result = $result"
    ;;
2)
    result=$((num1 - num2))
    echo "Result = $result"
    ;;
3)
    result=$((num1 * num2))
    ;;
4)
    result=$((num1 / num2))
    ;;
*)
    echo "Invalid choice"
    ;;
esac

-- INSERT --
```

1,5 Top

```
[root@ip-172-31-17-227 bin]# vi calculator.sh
[root@ip-172-31-17-227 bin]# chmod +x calculator.sh
[root@ip-172-31-17-227 bin]# ./calculator.sh
Simple Calculator
Enter first number: 40
Enter second number: 60
Choose an operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
Enter your choice (1-4): 3
Result = 2400
[root@ip-172-31-17-227 bin]# |
```

Explanation

- read -p – Accepts input from the user.
- case – Executes the selected arithmetic operation.
- \$(()) – Performs arithmetic calculations.
- if – Checks if the divisor is zero before division.
- echo – Displays the result or an error message.

8. Create a bash script to **check if a directory exists**, if not then create a directory.

```
#!/bin/bash

read -p "Enter the directory name: " dir

if [ -d "$dir" ]
then
    echo "Directory already exists."
else
    mkdir "$dir"
    echo "Directory created successfully."
fi

~
~
```

```
[root@ip-172-31-17-227 bashscript]# vi check_creat_dir.sh
[root@ip-172-31-17-227 bashscript]# chmod 755 check_creat_dir.sh
[root@ip-172-31-17-227 bashscript]# ./check_creat_dir.sh
Enter the directory name: raj
Directory already exists.
[root@ip-172-31-17-227 bashscript]# ./check_creat_dir.sh
Enter the directory name: raji
Directory created successfully.
[root@ip-172-31-17-227 bashscript]# ls
backup.sh      file1.txt  file4.txt  install_nginx.sh  raj
check_creat_dir.sh  file2.txt  file5.txt  install_tomcat.sh  raj_20260705_062716.tar.gz
dir.bash      file3.txt  file6.txt  multiple.sh       raji
[root@ip-172-31-17-227 bashscript]# |
```

9. Create a bash script to delete the last 3 lines of a file.

```
#!/bin/bash

read -p "Enter the file name: " file

if [ -f "$file" ]
then
    total_lines=$(wc -l < "$file")

    if [ "$total_lines" -ge 3 ]
    then
        head -n -3 "$file" > temp_file
        mv temp_file "$file"
        echo "Last 3 lines deleted successfully."
    else
        echo "The file has less than 3 lines."
    fi
fi
else
    echo "File does not exist."
fi

~
~
```

```
[root@ip-172-31-17-227 bashscript]# ./delline.sh
Enter the file name: moon.txt
Last 3 lines deleted successfully.
[root@ip-172-31-17-227 bashscript]# cat moon.txt
The Moon is Earth's only natural satellite.
It revolves around the Earth and is the fifth-largest natural satellite in the Solar System.
The Moon orbits the Earth.
It takes about 27.3 days to complete one orbit around the Earth.
The Moon does not produce its own light.
It appears bright because it reflects sunlight.
The Moon's gravity affects Earth.
Its gravitational pull causes ocean tides and influences Earth's rotation.
The Moon has no atmosphere.
Since it has almost no atmosphere, there is no weather, wind, or rain on the Moon.
The Moon has many craters.
[root@ip-172-31-17-227 bashscript]# |
```

Explanation

- `read -p` – Accepts the file name from the user.
- `-f` – Checks whether the file exists.
- `wc -l` – Counts the total number of lines in the file.
- `head -n -3` – Prints all lines except the last 3.
- `>` – Redirects the output to a temporary file.
- `mv` – Replaces the original file with the updated file.
- `echo` – Displays the success or error message.

10. Bash script to monitor cpu and if it is more than 80% then send email notification.

Objective: To create a Bash script that monitors CPU usage. If the CPU utilization exceeds 80%, the script automatically sends an email notification to the administrator.

Purpose: The purpose of this script is to continuously monitor CPU utilization and notify the system administrator when CPU usage exceeds the defined threshold.

This helps in identifying performance issues early and enables timely troubleshooting.

```
#!/bin/bash

THRESHOLD=80
EMAIL="brajkumari7675@gmail.com"

CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print int($2 + $4)}')

if [ "$CPU_USAGE" -gt "$THRESHOLD" ]
then
    echo "Warning! CPU usage is ${CPU_USAGE}% on $(hostname)." | \
    mail -s "High CPU Usage Alert" "$EMAIL"

    echo "Email notification sent."
else
    echo "CPU usage is normal: ${CPU_USAGE}%"
fi
~
~
```

Here, install Mail Utility

dnf install -y mailx

```
[root@ip-172-31-17-227 bashscript]# chmod 755 cpu_monitor.sh
[root@ip-172-31-17-227 bashscript]# dnf install -y mailx
Amazon Linux 2023 Kernel Livepatch repository          34 kB/s | 2.9 kB      00:00
Dependencies resolved.
=====
Package                Architecture      Version              Repository          Size
=====
Installing:
mailx                   x86_64            12.5-43.amzn2023.0.1 amazonlinux          263 k
Transaction Summary
=====
Install 1 Package

Total download size: 263 k
Installed size: 497 k
Downloading Packages:
mailx-12.5-43.amzn2023.0.1.x86_64.rpm                  7.2 MB/s | 263 kB      00:00
-----
Total                                                    3.0 MB/s | 263 kB      00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                : 1/1
  Running scriptlet: mailx-12.5-43.amzn2023.0.1.x86_64 1/1
```

```
Installed:
  mailx-12.5-43.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-17-227 bashscript]# |
[root@ip-172-31-17-227 bashscript]# ./cpu_monitor.sh
CPU usage is normal: 3%
[root@ip-172-31-17-227 bashscript]# |
```

Explanation

- THRESHOLD=80 – Sets the CPU usage limit.
- EMAIL="brajkumari7675@gmail.com" – Specifies the recipient email address.
- top -bn1 – Retrieves CPU usage information.
- awk – Extracts and calculates CPU usage.
- if – Checks whether CPU usage exceeds the threshold.
- mail -s – Sends an email with the specified subject.
- echo – Displays status messages.

11. Bash script to monitor disk space and if it is more than 80% then send email notification.

Objective: To create a Bash script that monitors disk space usage. If the disk utilization exceeds 80%, the script automatically sends an email notification to the administrator.

Purpose: The purpose of this script is to monitor the disk usage of the file system and notify the administrator when the usage exceeds the defined threshold.

This helps prevent issues caused by insufficient disk space, such as application failures, logging problems, or system downtime.

```
#!/bin/bash
THRESHOLD=80
EMAIL="brajkumari7675.com"
DISK_USAGE=$(df -h / | awk 'NR==2 {gsub("%",""); print $5}')
if [ "$DISK_USAGE" -gt "$THRESHOLD" ]
then
    echo "Warning! Disk usage is ${DISK_USAGE}% on $(hostname)." | \
    mail -s "High Disk Usage Alert" "$EMAIL"
    echo "Email notification sent."
else
    echo "Disk usage is normal: ${DISK_USAGE}%"
fi
```



```
[root@ip-172-31-17-227 bashscript]# ./cpu_monitor.sh
CPU usage is normal: 3%
[root@ip-172-31-17-227 bashscript]# vi disk_monitor.sh
[root@ip-172-31-17-227 bashscript]# chmod 755 disk_monitor.sh
[root@ip-172-31-17-227 bashscript]# ./disk_monitor.sh
Disk usage is normal: 27%
[root@ip-172-31-17-227 bashscript]# |
```

Explanation

- THRESHOLD=80 – Sets the maximum allowed disk usage percentage.
- EMAIL="brajkumari7675@gmail.com" – Specifies the recipient email address.
- df -h / – Displays disk usage information for the root (/) file system.
- awk 'NR==2 {gsub("%",""); print \$5}' – Extracts the disk usage percentage and removes the % symbol.
- if – Checks whether disk usage exceeds the threshold.

- mail -s – Sends an email with the specified subject.
- echo – Displays the current status or confirmation message.

12. Bash script to monitor memory and if it is more than 80% then send email notification.

```
#!/bin/bash
THRESHOLD=80
EMAIL="brajkumari7675@gmail.com"
MEMORY_USAGE=$(free | awk '/Mem:/ {printf("%.0f"), $3/$2 * 100}')
if [ "$MEMORY_USAGE" -gt "$THRESHOLD" ]
then
    echo "Warning! Memory usage is ${MEMORY_USAGE}% on $(hostname)." | \
    mail -s "High Memory Usage Alert" "$EMAIL"
    echo "Email notification sent."
else
    echo "Memory usage is normal: ${MEMORY_USAGE}%"
fi
~
~

[root@ip-172-31-17-227 bashscript]# vi memory_monitor.sh
[root@ip-172-31-17-227 bashscript]# chmod 755 memory_monitor.sh
[root@ip-172-31-17-227 bashscript]# ./memory_monitor.sh
Memory usage is normal: 32%
[root@ip-172-31-17-227 bashscript]#
```

Explanation

- THRESHOLD=80 – Sets the maximum allowed memory usage percentage.
- EMAIL="admin@example.com" – Specifies the recipient email address.
- free – Displays memory usage information.
- awk '/Mem:/ {printf("%.0f"), \$3/\$2 * 100}' – Calculates the percentage of memory currently in use.

- if – Checks whether memory usage exceeds the threshold.
- mail -s – Sends an email with the specified subject.
- echo – Displays the current status or confirmation message.

```
[root@ip-172-31-17-227 bashscript]# vi test1.sh
[root@ip-172-31-17-227 bashscript]# chmod +x test1.sh
[root@ip-172-31-17-227 bashscript]# ./test.sh
bash: ./test.sh: No such file or directory
[root@ip-172-31-17-227 bashscript]# ./test1.sh
[root@ip-172-31-17-227 bashscript]# cat cron_output.txt
Cron Job Executed on Sun Jul  5 11:59:18 UTC 2026
[root@ip-172-31-17-227 bashscript]# |
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
0 0 * * * /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

Crontab: Crontab (Cron Table) is a Linux utility used to schedule and manage jobs (commands or scripts) that should run automatically at specific times or on recurring schedules.

Objective: To automate the execution of commands or scripts at specified dates and times without requiring manual intervention.

Purpose: The purpose of **crontab** is to schedule repetitive or time-based tasks automatically.

It helps system administrators and users perform routine activities such as backups, log cleanup, system monitoring, email notifications, software updates, and

service restarts at predefined intervals, improving efficiency and reducing manual effort.

Crontab Syntax

* * * * * command_to_execute

| | | | |

| | | | | _____ Day of Week (0–7) (0 or 7 = Sunday)

| | | | _____ Month (1–12)

| | | _____ Day of Month (1–31)

| | _____ Hour (0–23)

| _____ Minute (0–59)

Execute Each Exercise

1. April 5th Midnight

```
0 0 5 4 * /root/rajkumari/bashscript/test1.sh
~
~
~
~

[root@ip-172-31-17-227 bashscript]# crontab -l
0 0 5 4 * /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

2. 5th of January, June, November (if Thursday)

```
0 0 5 1,6,11 * /root/rajkumari/bashscript/test1.sh
~
~
~
~
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
0 0 5 1,6,11 * /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

3. At 05 and 27 Minutes of 9, 10, and 11 Hours Every Day

```
5,27 9-11 * * * /root/rajkumari/bashscript/test1.sh
~
~
~
~
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
5,27 9-11 * * * /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

4. 9:34 AM on 15th August

```
34 9 15 8 * /root/rajkumari/bashscript/test1.sh
~
~
~
~
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
34 9 15 8 * /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

5. Every Midnight

```
0 0 * * * /root/rajkumari/bashscript/test1.sh
~
~
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
0 0 * * * /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

6. Every Saturday at 11:59 PM

```
59 23 * * 6 /root/rajkumari/bashscript/test1.sh
~
~
~
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
59 23 * * 6 /root/rajkumari/bashscript/test1.sh
[root@ip-172-31-17-227 bashscript]# |
```

7. After Every Reboot

```
@reboot /root/rajkumari/bashscript/test1.sh
```

```
~  
~  
~
```

```
[root@ip-172-31-17-227 bashscript]# crontab -l
```

```
@reboot /root/rajkumari/bashscript/test1.sh
```

```
[root@ip-172-31-17-227 bashscript]# |
```

```
[root@ip-172-31-17-227 bashscript]# systemctl status crond
```

```
● crond.service - Command Scheduler  
   Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; preset: enabled)  
   Active: active (running) since Sun 2026-07-05 03:33:12 UTC; 9h ago  
     Main PID: 1865 (crond)  
       Tasks: 1 (limit: 1059)  
      Memory: 2.2M  
         CPU: 20.884s  
    CGroup: /system.slice/crond.service  
            └─1865 /usr/sbin/crond -n
```

